

《软件度量及应用》

课程项目报告

项目名称 Merlin Garden 自动度量平台

专业班级 软件工程 1905，软件工程 1906

学号	姓名	职责
8209190507	夏家豪	组长，整体算法与模块设计， 前端开发
8209190515	周禄庭	用例图、控制流图后端算法 设计，相关数据准备
8209190529	王璐娜	代码行解析后端算法设计， 相关文档编写
8209190624	郭宇轩	类图后端算法设计，通信接 口负责
8209190625	刘梓康	后端整体开发，AST 后端算法 负责

批阅教师：杨柳

2022 年 04 月 18 日

《自动化度量工具设计——Merlin Garden》

实验学时： 8 实验地点： 二综 实验日期： 2022/04/18

一、实验目的

本实验课程是软件工程专业学生的专业选修课程《软件度量及应用》的实验课。通过实验，帮助学生掌握软件度量的有关概念、原理、方法和技术，了解软件度量发展趋势和工程中的应用，提高学生运用有关工具进行软件度量的能力；培养学生运用工程化思想解决实际问题的能力；实验报告的编写、总结实验结果的能力。授课对象为软件工程专业本科学生。

度量在成功的软件开发过程中起到了非常关键的作用，也是软件工程作为一门真正的工程学科的基础。本课程系统介绍软件度量的原理、方法和工具，叙述软件度量在软件产品和软件过程的开发、评价（评估）和改进等方面的应用，主要包括软件度量的基本概念；产品内部属性度量；产品外部属性度量；度量系统的设计和度量结果的分析及应用等方面的基本原理。

二、实验内容

本实验课的重点是应用工程化的原理和方法开发软件度量工具并进行度量分析应用的能力培养。

（一）实验方式主要有：

1. 由指导老师讲解实验的基本要求，完成的任务操作要领及注意事项。
2. 实验 3-5 人一组。
3. 学生在完成预习报告后才能进入实验室进行实验。

（二）基本要求为：

1. 了解软件度量的基本概念和功能。
2. 熟练掌握软件度量工具使用和基本方法步骤。
3. 通过实验进一步掌握运用软件度量工具解决软件开发中有关问题的方法。
4. 学会使用工具进行项目和实验报告的文档写作。

（三）项目总体要求

开发一个中小型软件度量自动化工具，把软件度量课程和其他相关课程的基本原理和方法应用到软件项目实际开发中。要求：

1、项目有创意，开发工具和开发模型不限，项目应该覆盖大部分软件度量课程知识，能够利用不同软件度量方法对软件设计开发各阶段中的不同软件实体进行度量。

（1）项目主要实现面向对象度量，程序实现 LK 度量模型与 CK 度量模型中的相关度量点，其中可加入圈复杂度、代码行的度量等传统的度量点；

（2）可以加入其它度量方法：功能点度量、特征点度量、用例度量、面向对象度量、复杂性度量等；

（3）度量的软件实体：软件规模（LoC），工作量、成本、开发时间、人员等。

（4）输入项包括：类图、程序流程图、用例图等设计图，程序代码，用户输入等。

（5）建议：对于 Java 程序，可以利用 Eclipse 中的 ASTParser 得到程序代码的 AST（抽象语法树），再进行统计分析。

2、项目文档必须规范，要求内容充实完整。

（四）项目总体要求

报告名称	内容要求
项目设计与实现报告	应突出如何进行利用度量模型或者利用程序环境中的接口或插件实现软件度量功能； 功能需求、数据需求、非功能需求 功能需求采用数据流图或用例图描述，数据需求采用 ER 图或 UML 类图描述，非功能需求采用文字或其他工具描述 设计系统的体系结构，对重要模块进行详细描述，建议采用 UML 描述设计相应的自动化度量方案与算法
项目分析报告	展现自动化度量工具的功能，即自动化度量工具的实现结果 利用度量软件或者度量工具进行预测、评估以及改进时，与实际应用进行比较，分析度量工具的准确性、实用性和有效性。

三、实验方法

1. 代码行分析：利用 python 直接读取分析代码文件
2. UML 系列图：利用 powerdesign 导出 xml 文件或者 oom，对 xml 或者 oom 文件解构统计解析
3. 控制流图：利用 visio 导出 xml 文件，对 xml 文件解构统计解析
4. 代码逻辑分析：利用抽象语法树，将代码转换为语法树，再进行解析
5. 主观因素计算：前端通过填表的形式填入直接计算
6. 前端展示：利用 nuxt 框架与 antdesignvue 组件，前后端分离快速开发，并通过 axios 建立前后端通信，存取后端数据

四、实验步骤

（一）前端与项目整体设计（夏家豪）：

1. 本项目采用前后端分离的方式开发，在设计完整体项目架构和模块后，前后端同步进行开发。前端采用 nuxt 框架与 ant-design-vue 组件库，界面设计简洁清晰，同时可以利用 axios 进行前后端的通信，方便数据存取。

2. 由于功能点度量与用例点度量里有大量涉及到主观判断的内容，例如功能点度量的 UFC 与 VAF 以及用例点度量的 EF、TCF，这些都是主观的，只能通过填写表格或表单的方式直接计算。

3. 部分指标例如用例点度量的 UUC 和 UAW，这两个虽然是客观可分析的，但是如果只看用例图要判断角色的类型，要涉及到利用深度学习的语义识别才能判断是人、系统还是通信。UUC 虽然可以利用代码去检测内部是涉及到多少方法或实例，但是在前期设计阶段是不会有代码的，所以也只能填数据。

4. 类图、用例图、控制流图、代码辅助这四个模块都涉及到文件传输，前端需要提供一个文件上传的功能，用户将规定格式的文件传上去后，点击分析可以得到需要的结果。

5. 所有前端页面写完后，将接口准备好，每写完一个后端算法就进行对接与调整。

（二）抽象语法树分析代码复杂度设计（刘梓康）

1. 实验中抽象语法树 (AST) 的解析是面向对象软件度量的核心。因为 Java 语法的复杂性，我们在解析语法时设计了一些算法来保证结果的准确性。实验使用 Python 语言进行，利用 Python 的第三方 Java 代码解析库 javalang 对 Java 文件的语法和语义进行分析。

2. 语句分类解析

对于 Java 语法，我们充分考虑了类定义、内部类、方法声明、属性声明、局部变量声明、This 指针等声明性语法，同时加入了各种控制流语句的解析，比如条件语句、循环语句、try-catch 异常捕获语句、普通代码块（使用“{}”包裹）、操作符连接的布尔值语句（如 item == null）等。解析时使用队列结构对抽象语法树进行遍历和计算，对各种常见的 Java 语法均有支持。

3. 变量局部作用域与全局作用域问题

在计算 LCOM 时，Java 语句的局部作用域对变量引用的统计有着一定影响，比如在条件语句中声明了一个与类属性重名的局部变量，那么此时的属性引用不应该计算在 LCOM 中，而在条件语句之外对类属性的引用则应该计算在 LCOM 中。此外，条件语句、普通代码块和循环语句都有着可以互相嵌套的特点，对于不同位置声明的局部变量，其具体作用域也不同。经过不断地思考，我们找到了解析变量不同作用域的等价算法，即使用堆栈+队列组合结构进行语法解析。

当语法解析树遇到一个语句块（条件、循环、try-catch 异常捕获、普通代码块等）时，新建一个队列并且将该代码块输入进新队列中，然后将新队列入栈。这样，解析算法会对代码块的局部作用域进行解析，每当遇到引用对象时，从栈顶开始搜寻，直到栈的底端，层层向上判断引用对象是局部作用域声明的变量还是类的属性。当局部代码块解析完毕时，即栈顶的队列为空时，将该空队列出栈，并找到栈中的下一个元素，即为代码的更外层作用域，如此往复，直到栈为空为止。该流程可以被描述为如下算法：

算法：变量作用域的语义解析

输入：语法树

输出：代码解析结果

初始化：空队列 Queue1、空栈 Stack

流程：

```
第一个语句入队列 Queue1
Queue1 入栈 Stack
While Stack 不为空 Then
    获取栈顶队列 Q
    While Q 不为空 Then
        获取 Q 队列首语句 state
        If state 为代码块 Then
            新建队列 Q2
            state 入队列 Q2
            Q2 入栈 Stack
            跳出内层 while 循环
        End If
        If state 为引用变量 Then
            检查队列的每一个元素，判断是否为局部变量引用
        End If
    End While
    栈 Stack 弹出顶端空队列
End While
Return 解析结果
```

3. LCOM 与 RFC 的计算

根据类中每个方法的属性引用与方法调用分析结果，进行计数统计即可。

（三）用例点度量与程序复杂度分析设计（周禄庭）

1. 用例点估算：用例图中包含的角色越多，用例越多，说明系统的功能越多，项目也就越复杂。针对用例图的.oom 文件进行分析。

有效信息：

执行者：

```
<c:Actors>
<o:Actor Id= “__” >
<a:Name>执行者 1 名称</a:Name>
<a:Code>执行者 1 代码（与名称相同）</a:Code>
</o:Actor>
<o:Actor Id= “__” >
<a:Name>执行者 2 名称</a:Name>
<a:Code>执行者 2 代码（与名称相同）</a:Code>
</o:Actor>
</c:Actors>
```

用例：

```
<c:UseCases>
<o:UseCase Id= “__” >
<a:Name>用例 1 名称</a:Name>
<a:Code>用例 1 代码（与名称相同）</a:Code>
</o:UseCase>
<o:UseCase Id= “__” >
<a:Name>用例 1 名称</a:Name>
<a:Code>用例 1 代码（与名称相同）</a:Code>
</o:UseCase>
</c:UseCases>
```

2. McCabe 圈复杂度的计算：由于传输的为 oom 文件，计算圈复杂度的方法就可以是将判断节点个数计算出来后直接+1。为了验证图的正确性（防止不符合规范），就进一步计算出了其它节点的数量以及边的数量，通过“边数-节点数+2”的方法算出圈复杂度并和刚才才计算出的圈复杂度进行比较，如果正确说明图是正确的，可以计算出有效的圈复杂度。

针对控制流图的.oom 文件进行分析

有效信息：

操作：

```
<c:Activities>
<o:Activity Id= “操作 1Id” >
<a:Name>操作 1（活动）名称</a:Name>
<a:Code>操作 1（活动）代码</a:Code>
</o:Activity>
<o:Activity Id= “操作 2Id” >
<a:Name>操作 2（活动）名称</a:Name>
<a:Code>操作 2（活动）代码</a:Code>
</o:Activity>
</c:Activities>
```

判断：

```
<c:Decisions>
<o:Decision Id= “判断 1Id” >
<a:Name>判断 1 名称</a:Name>
```

```

        <a:Code>判断 1 代码</a:Code>
    </o:Decision>
    <o:Decision Id= “判断 2Id” >
        <a:Name>判断 2 名称</a:Name>
        <a:Code>判断 2 代码</a:Code>
    </o:Decision>
</c:Decisions>

```

边:

```

<c:Flow>
<o:ActivityFlow Id = “连接线 1Id” >
<c:Object1>
    <o:Start/End/Decision/Activity Ref= “对应 Id” />
</c:Object1>
<c:Object2>
    <o:Start/End/Decision/Activity Ref= “对应 Id” />
</c:Object2>
</o:ActivityFlow>
</c:Flows>

```

(四) 利用类图 LK、CK 度量设计 (郭宇轩)

实验中识别类图的方法基于 XML 文件展开。

1. 首先需要读取 XML 文件中的信息。

类图中类的格式大致如下:

```

<packagedElement xmi:type="uml:Class" xmi:id="_t5lVEL_DEeytCevADA78fA" name="Image"
clientDependency="_uFNWQL_DEeytCevADA78fA">
    <ownedAttribute xmi:id="_t9PGAL_DEeytCevADA78fA" name="image"
visibility="private" type="_t5lVEL_DEeytCevADA78fA"/>
    <ownedOperation xmi:id="_t7pxoL_DEeytCevADA78fA" name="change"
visibility="public" bodyCondition="_t7yUgL_DEeytCevADA78fA">
        <ownedParameter xmi:id="_t8I50L_DEeytCevADA78fA" name="change"
type="_t8PngL_DEeytCevADA78fA" direction="return"/>
    </ownedOperation>
</packagedElement>

```

其中 packagedElement 指代某种类、接口或是关系的信息, 我们通过 xmi:id 识别其具体类别。在类或接口中, ownedAttribute 指类中单个属性, ownedOperation 指类中单个方法, ownedParameter 指方法中单个参数, 其中包含 direction="return" 的表示方法返回类型, 通过这些基本标签, 我们便可以获取类中所有信息。

Python 中自带相关的库, 可以提供读取文件, 将内容转化为 doc 格式的功能。在转化为 xml 的格式后, 我们读取其中的标签, 获取便签内的信息。其中类、接口由同一种标签定义。先获取类中的继承关系并保存, 从而判明父类子类关系; 然后再去存储类名、类中属性、类中方法等相关信息; 最后再对具备专门标签的关联关系、实现关系等特别分析, 进行存储。

2. 分析、计算、获取所需数据。类图中主要是获取各类的 CS, NOO, NOA, DIT, NOC, CBO。由于读取信息中已经获取到类的父类、类名、类中属性、类中方法, 我们可以很快速计算得到 CS, 然后根据有无父类决定 NOO、NOA 是否需要计算。如果需要计算, 则寻找与父类方法名字、参数列表、返回类型一致、也就是重载的方法数量, 获取 NOO, 然后把方法数减去 NOO 获取 NOA。

此外，根据继承关系，设置一个树，获取各类的 DIT，另外根据该树获取 NOC，最后寻找关联关系、实现关系等关系判断耦合度，也就是 CBO

3. 问题与修正方法

Q1: 重载判断通过对照方法名、参数列表以及返回类型，但由于参数列表有时会调整顺序，可能会出现只是调转了顺序，于是不被视为重载了。

A1: 参数列表默认按照首字母排序

Q2: 前端传来的不是文件，而是文件内容，不适合使用读取文件格式的方法

A2: 将文件格式转化为字符串类型后，再转化字符串为 xml 格式，最后读取

（五）代码行度量统计设计（王璐娜）

代码行的统计主要考虑的是代码本身行数以及是否为注释的区分。我们利用正则表达式与堆栈的方式进行了统计，考虑到如果有注释就只能是：

```
a--;
a--; //b
//b
a--; /*b
c
d*/
/*b
d*/
```

这几种情况，所以其实统计起来并不难。而对于存在空行的统计，我们也单独将每一行做 strip 处理，将只包含空格、\t 和 \n 的行变为空。在统计过程中由于对 PLOC 和 LLOC 的认识不足一开始出现了一些错误，仔细研究区分后发现物理行是程序语句在编辑器中的占用行，而逻辑行指的是程序语句，逻辑行是物理行，但是一个物理行可以包含多个逻辑行。

五、实验结果

（一）功能点度量

功能点度量

类图辅助度量

用例图辅助度量

控制流图辅助度量

代码辅助度量

UFC计算

请在表格中填入对应的个数	Simple	Average	Complex	编辑
External Inputs	0	2	0	Edit
External Outputs	0	3	0	Edit
External Queries	0	2	0	Edit
External Interfaces Files	0	2	0	Edit
Internal Logic Files	0	1	0	Edit

计算得到UFC为55

VAF计算

序号	系统特征	简单描述	等级	编辑
1	数据通讯	软件系统的信息交互情况	3	Edit
2	数据分布处理	软件系统如何处理分布数据	3	Edit
3	系统性能	用户对软件系统的响应时间和吞吐量的要求	0	Edit

4	使用配置	当前硬件平台的使用度	5	Edit
5	事务处理频率	每日、每周以及每月等的事务处理情况	0	Edit
6	在线数据输入	在线信息输入的百分比	3	Edit
7	操作便利	软件系统是否为用户操作提供便利	3	Edit
8	在线更新	多少LFI通过在线事务更新	3	Edit
9	处理复杂度	软件系统是否广泛使用逻辑和数学运算	0	Edit
10	复用性	本软件系统是否为多个应用系统复用	5	Edit
11	安装难易	版本维护和安装的难易程度	0	Edit
12	运行维护	软件系统的启动、备份以及恢复的难易程度	0	Edit
13	多站点支持	软件系统是否用于多组织和多站点	0	Edit
14	变更支持	软件系统对查询、统计以及变更维护的支持	3	Edit
计算得到VAF为0.93				
该项目功能点度量结果FP为51.16				

（二）类图辅助度量

功能点度量

类图辅助度量

用例图辅助度量

控制流图辅助度量

代码辅助度量

文件上传处

选择要上传的类图(xml格式)

开始分析

@ 1.xml

分析结果展示

类名	类规模度量CS	方法重写数NOO	增加方法数目NOA	子类数目NOC	继承树深度DIT
Image	1	0	0	0	0
Template	14	2	10	2	1
Object	1	1	0	1	0
Client	1	0	0	0	0
ShadowTemp	1	0	0	0	2
DeepTemp	1	0	0	0	2
Cloneable	0	0	0	0	0
Serializable	0	0	0	0	0

<

1

>

（三）用例点度量

1. 用例与角色填写错误警告

UUC计算

简单用例个数

8

普通用例个数

0

复杂用例个数

0

请检查用例个数是否填写错误,图中用例为13个

该用例图所表示的UUC为0

UAW计算

简单角色个数

0

普通角色个数

5

复杂角色个数

0

请检查角色个数是否填写错误,图中角色为4个

该用例图所表示的UAW为8

2. 正常计算

功能点度量

类图辅助度量

用例图辅助度量

控制流图辅助度量

代码辅助度量

文件上传处

选择要上传的用例图(oomt格式)

开始分析

@ CCMS.com

UUC计算

简单用例个数

2

普通用例个数

9

复杂用例个数

2

该用例图所表示的UUC为130

UAW计算

简单角色个数

1

普通角色个数

3

复杂角色个数

0

该用例图所表示的UAW为7

TCF计算

技术复杂性Ti	说明	权重WF	等级	编辑
T1	分布式系统	2	0	Edit
T2	响应吞吐量性能	1	3	Edit
T3	终端用户效率	1	5	Edit
T4	复杂的内部结构	1	0	Edit
T5	代码必须重用	1	0	Edit

T6	易安装性	0.5	0	Edit
T7	易用性	0.5	1	Edit
T8	可移植性	2	0	Edit
T9	易更改性	1	0	Edit
T10	并发性	1	0	Edit
T11	特殊的安全性	1	0	Edit
T12	提供第三方接口	1	0	Edit
T13	需要特别的用户培训	1	1	Edit

计算得到TCF为0.695

EF计算

环境复杂性Fi	说明	权重	等级	编辑
F1	熟悉UML的程度	1.5	0	Edit
F2	开发应用程序的经验	0.5	0	Edit
F3	面向对象的经验	1	0	Edit
F4	主分析师的能力	0.5	0	Edit
F5	激励机制	1	0	Edit
F6	需求稳定度	2	0	Edit
F7	具有兼职人员	-1	2	Edit
F8	具有复杂编程	-1	0	Edit

计算得到EF为1.46

计算得到UCP为139.02

（四）控制流图辅助度量

♡ 功能点度量 ☒ 类图辅助度量 🔍 用例图辅助度量 — 控制流图辅助度量 ☆ 代码辅助度量

文件上传处

⬇ 选择要上传的控制流图(oom格式)

@ ControlFlowDiagram.oom

开始分析

经过分析,该控制流图的McCabe圈复杂度为4

（五）代码辅助度量

♡ 功能点度量 ☒ 类图辅助度量 🔍 用例图辅助度量 — 控制流图辅助度量 ☆ 代码辅助度量

文件上传处

⬇ 选择要上传的代码文件(java格式)

@ Member.java

开始分析

代码行分析结果

该java文件中,注释代码行共1行,非注释代码行共23行,物理代码行共28行,逻辑代码行共24行
代码注释密度为0.05

抽象语法树分析结果

类名	类响应数量RFC	类缺乏内聚性LCOM
[-] Member	5	1
方法名	调用的外部方法	使用的变量
createMember	r.nextInt	name, phoneNum, password, ID
modifyInfo		name, phoneNum, password
readOrder	order.getOrderList	ID

< 1 >

六、实验结论

（一）功能点度量

该点测试时，我们采用了老师 ppt 上的例子，计算结果与老师的结果一致

Function Points: example

- $N_f=2$: document filename, personal dictionary-name
- $N_o=3$: misspelled word report, number-of-words-processed message, number-of-errors-so-far message
- $N_g=2$: words processed, errors so far
- $N_{ef}=2$: document file, personal dictionary
- $N_{df}=1$: dictionary

$$\text{UFC} = 4 \times 2 + 5 \times 3 + 4 \times 2 + 7 \times 2 + 10 \times 1 = 55$$

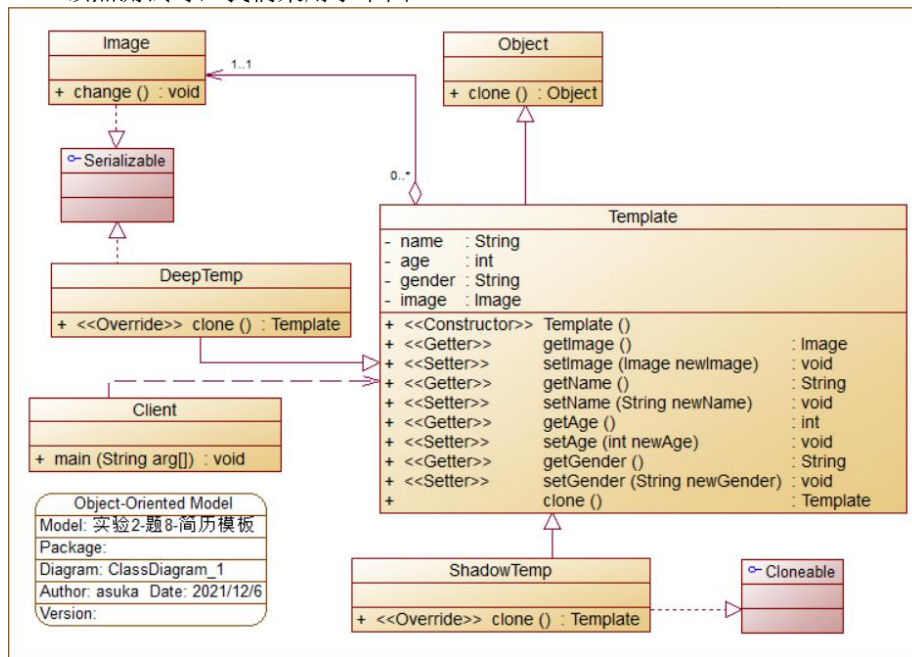
- Suppose that
F1=F2=F6=F7=F8=F14=3; F4=F10=5

$$\text{VAF} = 0.65 + 0.01(18+10) = 0.93$$

$$\text{FP} = 55 \times 0.93 \approx 51.16$$

(二) 类图辅助度量

该点测试时，我们采用了下图



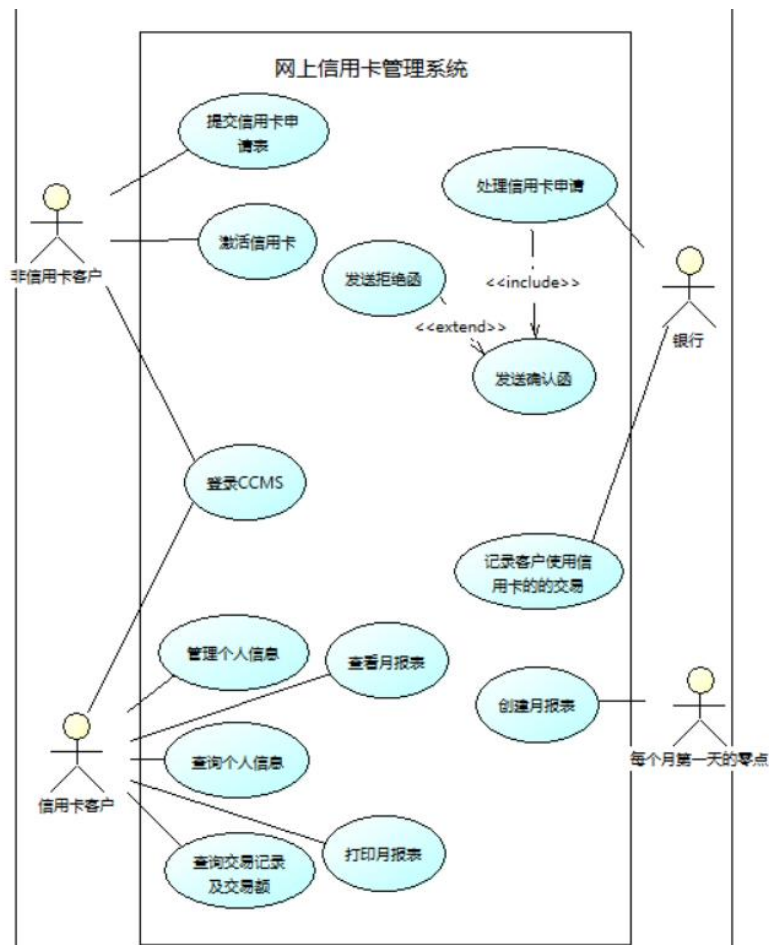
统计结果与手工计算无误

```

1:02:02
按照你发的顺序:
1 0 0 0 2
14 1 9 1 2 2
1 0 0 0 1 0
1 0 0 0 1
1 1 0 2 0 1
1 1 0 2 0 1
0 0 0 0 1
0 0 0 0 2
    
```

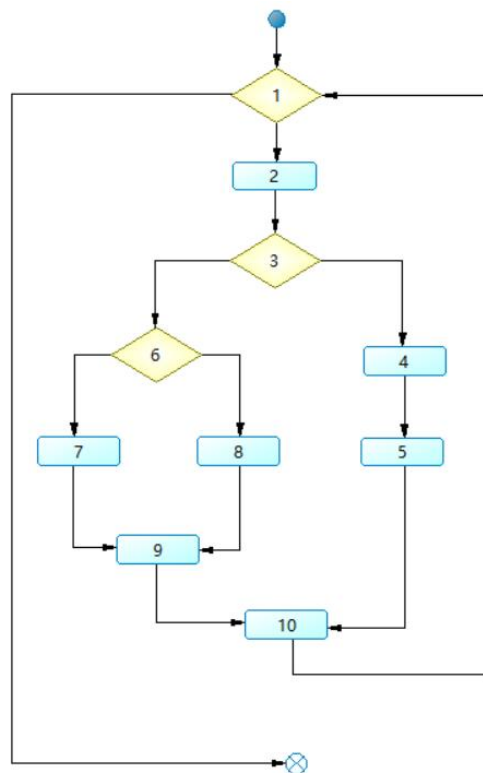
(三) 用例点度量

该点测试时我们采用了下图进行测试，用例与角色个数统计结果与手工计算一致



(四) 控制流图辅助度量

该点测试时我们采用了下图进行测试，圈复杂度结果与手工计算一致 ($14-12+2=4$)



(五) 代码辅助度量

该点测试时我们采用了如下代码进行测算，与手工计算结果一致

```
1 import java.util.Random;
2
3 public class Member{
4     private String ID;
5     private String name;
6     private String phoneNum;
7     private String password;
8     //435
9     private void createMember(){
10         Random r = new Random();
11         ID = r.nextInt(100000);
12         name = "lily";
13         phoneNum = 123456;
14         password = 111111;
15     }
16
17     private void modifyInfo(){
18         name = "kiki";
19         phoneNum = 321654;
20         password = 222222;
21     }
22
23     private void readOrder(){
24         Order order = new Order();
25         order.getOrderList(ID);
26     }
27
28 }
```

七、实验小结

夏家豪：在这次项目中我担任的是组长，在看到实验指导书后我就直接找到了一起组队的同学然后开始讨论这个项目该做什么以及如何开发这个项目。首先，根据老师指导书中写到的“建议用抽象语法树”，我们讨论了满足这次实验的最基本任务就是用抽象语法树来对代码进行解析，有代码的情况下我们可以直接对代码行进行统计分析，也可以利用语法树来判断一个代码文件内的类、方法、变量的属性以及相互的嵌套调用关系。但是我考虑到一般对于项目的度量是用来估算项目以及管控项目的，如果只是对后期才会出现的代码进行度量的话，并不能体现出度量的优势，我们更应该在早期就对一个项目进行分析，这个时候就会涉及到图。一张标准的类图、用例图、控制流图等是很能表示出一个项目早期设计的内容的。但是这个时候我们就开始纠结图片能否被我们读取。我认为如果是用 powerdesign 或者 visio 去画图，他保存的源文件一定是能够解析的，如果能掌握源文件的解析方法就能读取图内的元素。同时我也考虑到如果能导出 xml 的话，xml 可以更加直观看出元素关系。回寝室后我惊喜发现这两个软件都能导出 xml 文件，且原本保存的 oom 文件本身也是一种特殊的 xml，所以我们直接对这两类文件展开了设计。我结合不同的图能够得到的度量指标对组员进行分工，同时对其中几个算法进行了粗略的构思设计，小组成员很快就完成了整个项目后端部分的开发，经过一天的对接和调试，项目在实验课前就已完成。通过这次实验，我对软件度量的理解更加深刻了。

刘梓康：这次实验我对 Java 语法的解析有了更深刻的认识，同时对面向对象软件度量的一些度量点和度量方法理解更清晰，并在实验的过程中不断明白了各个度量点的意义和实用价值。

周禄庭：在开发过程中遇到了一些问题，在这里列举（1）用例图：由于未调整前的角色权重 UAW 和未调整前的用例权重 UUC 都需要进行主观判断，因此不能将其从图中量化，无法从图中读取出来。解决方法之一是通过机器学习训练模型通过用例名称识别其角色权重（有接口 API 的外部系统、通过协议交互的外部系统以及通过人机界面交互的人）。但是由

于时间原因就没来得及开发。因此，使用了另一种方法，由于可以计算出来图中执行者的数量和用例数量，就通过在前端输入用例图中三种执行者的数量进行分析；并且可以通过算出总的执行者数量防止用例图过大时用户查错具体的角色类别个数。同理，UUC 也通过用户输入来确定每个角色类别的具体个数（简单、普通、复杂），并且通过对图中用例数的分析保证总体用例个数正确。然后，将具体的 UAW 和 UUC 计算得出得到未调整前的用例点 UUCP，再计算技术复杂性因子 TCF，以及环境因子 EF，并计算出调整后的用例点 UCP，再通过 UCP 和工作量的比例计算出估算工作量。（2）前后端接口对接时，最开始的时候使用了 flask_uploads 插件，导致只能在前端使用 html 的 input 标签时才可以获取数据；而前端的框架为 nuxt，全部都是 js 写的，因此不能直接传输。在修改后，后端直接使用 flask 原生的文件传输方法获取到文件，并且可以逐行对文件内容进行处理。

郭宇轩：通过本次实验，我搞明白了 PowerDesigner 中类图的 XML 文件的构造，了解了基本的读取 XML 文件方法，还复习了一遍 LK 和 CK 内部度量项，加深了对于这些面向对象的度量项的了解。同时，我搞明白了重载的具体判断方法，保证了 NOO 的正确性，并且纠正了我的一些错误观点。

王璐娜：关于代码行的度量，我遇到的问题在于对 python 语言中字符串的处理不熟悉，需要一边上网查一边写代码。并且对于 PLOC 和 LLOC 的概念不清楚，后来知道它们分别表示物理代码行和逻辑代码行。最终通过对代码文件进行逐行处理得到 LOC 的相关值。在本次实验中，我尝试了利用时序图进行 RFC 的度量。类的响应数量 RFC 是对类复杂性和类之间耦合度的一种度量，通过计算一个类的对象对接受的消息进行响应的方法集合的数目得到，既包括类本身的方法，继承来的方法，还包括该类中方法调用的其它方法。实现的难点在于对时序图 xml 进行解析，这里使用 `xml.dom.minidom`，通过 `SXelement[i].getAttribute("xmi:type") == "uml:Actor"` 和 `SXelement[i].getAttribute("xmi:type") == "uml:Interaction"` 等得到元素对象，得到了时序图中的所有事件的内容后开始计算每个类调用的方法的集合。这个过程比较复杂，也比较容易出错。